DELIVEZ

Admin Backend — Route Reference

Where the admin API lives, every route, who can call it, what it checks, and what it changes.

Delivez Partners monorepo · core-api (NestJS 11) · Status as of 20 Oct 2026 (Steps 1–9 complete) · Generated from the source code

Contents

1. Where the code is
2. How a request reaches an admin route
3. Logging in to the admin console
4. Admin roles and permissions
5. City scope
6. Route summary
7. Route details — read routes
8. Route details — document review
9. Route details — store lifecycle
10. Store lifecycle state machine
11. Approval checks (LIVE / LIMITED)
12. What every write records
13. Error codes
14. Database tables used
15. Admin web pages → API routes
16. Seeded admin accounts (dev)
17. Tests
18. Known gaps and notes

1. Where the code is

The admin backend is a NestJS module inside the API app. Every admin route is served by one controller.

```
delivez_parteners/
└─ apps/
   ├─ core-api/                        ← the backend (NestJS)
   │  └─ src/
   │     ├─ main.ts                    ← global prefix "v1", Swagger at /docs, port 4000
   │     ├─ app.module.ts              ← registers AdminModule
   │     └─ modules/
   │        ├─ admin/                 ← ★ ADMIN BACKEND
   │        │  ├─ admin.module.ts       (13 lines)  wires controller + services
   │        │  ├─ admin.controller.ts   (140 lines) all /v1/admin/* routes
   │        │  ├─ admin.guard.ts        (57 lines)  AdminGuard, @AdminRoles, ROLES groups
   │        │  ├─ admin.service.ts      (270 lines) pipeline, store detail, document review
   │        │  └─ store-lifecycle.service.ts (156 lines) status transitions + approval checks
   │        ├─ identity/               ← OTP login, JWT, AuthGuard, @Apps decorator
   │        ├─ onboarding/             ← readiness score, required documents (used by admin)
   │        └─ files/                  ← signed document view URLs (/v1/files/raw)
   └─ admin-web/                       ← the admin console frontend (React, port 5174)
      └─ src/screens/  Login · Layout · Pipeline · Documents · StoreReview
packages/db/prisma/schema.prisma       ← AdminUser, AdminAction, AuditLog, PartnerDocument …
```

Base URL (local dev): `http://localhost:4000/v1/admin/…` · **Interactive API docs:** `http://localhost:4000/docs` (Swagger, tag "admin").

2. How a request reaches an admin route

Every `/v1/admin/*` request passes these checks, in this order. If any check fails, the route never runs.

Step	Where	What it does
1. AuthGuard (global)	`identity/auth.guard.ts`	Requires `Authorization: Bearer <access token>`. Verifies the RS256 JWT, rejects revoked sessions (logout), and sets `req.user`.
2. @Apps('ADMIN')	`admin.controller.ts` (class level)	The token's `app` claim must be `ADMIN`. A Partners-app token gets `403 WRONG_APP`.
3. AdminGuard	`admin/admin.guard.ts`	Loads the caller's `admin_users` row. It must exist and be active (`403 NOT_AN_ADMIN`). The admin's role must be in the route's allowed list (`403 ROLE_NOT_ALLOWED`). It then sets `req.admin = {adminId, userId, role, cityIds}`.
4. Validation	`ZodValidationPipe`, `ParseUUIDPipe`	IDs must be UUIDs. Query strings and bodies are validated with zod (`400 VALIDATION_FAILED`).
5. City scope	`assertCityScope()` in the services	City-scoped admins can only touch stores in their cities (`403 CITY_OUT_OF_SCOPE`).

Errors use one shape everywhere: `{ "error": { "code", "message", "details?", "correlationId" } }`.

3. Logging in to the admin console

Admins use the same phone + OTP login as partners, but ask for an `ADMIN` token. These routes live in `identity/auth.controller.ts`, not in the admin module.

Route	Details
`POST /v1/auth/otp/request` public	Body `{ phone }`. Sends a 6-digit code, valid for 5 minutes. Limits: one resend every 30 s, at most 5 codes per number per hour (`429 OTP_RATE_LIMITED`). In dev, the response includes `devCode`.
`POST /v1/auth/otp/verify` public	Body `{ requestId, code, app: "ADMIN" }`. Allows at most 5 wrong attempts. With `app: "ADMIN"`, the number must belong to an active `admin_users` row, or the request fails with `403 NOT_AN_ADMIN`. Returns `accessToken` (15 min) and `refreshToken` (30 days).
`POST /v1/auth/refresh` public	Body `{ refreshToken }`. Rotates the refresh token; an old one is rejected after use.
`POST /v1/auth/logout`	Revokes the current session, so its access token stops working immediately.

4. Admin roles and permissions

Roles come from the `AdminRole` enum. Permission groups are defined once in `ROLES` in `admin.guard.ts`. A route without `@AdminRoles(…)` defaults to **READ**.

Role	Read (view)	Verify docs	Approve / reject stores	Suspend	Reinstate

FOUNDER	✓	✓	✓	✓	✓
HEAD_OF_OPS	✓	✓	✓	✓	✓
REGIONAL_HEAD	✓	—	✓	✓	✓
CITY_OPS	✓	—	✓	—	—
COMPLIANCE	✓	✓	✓	✓	—
FINANCE	✓	—	—	—	—
LEGAL	✓	—	—	—	—
SUPPORT	✓	—	—	—	—
AUDITOR	✓	—	—	—	—
SALES_AGENT	—	—	—	—	—

"Approve / reject stores" covers start-review, approve-live, approve-limited, request-changes and reject. SALES_AGENT exists in the schema but is in no group yet, so it cannot use any admin route (see §18).

5. City scope

Each admin has `cityIds`. **Empty means all cities.** A scoped admin (e.g. the seeded "Bengaluru City Ops") is limited as follows:

- **Pipeline and document queue:** show stores in the admin's cities, plus stores that have no city yet.
- **Store detail, document review and all lifecycle actions:** call `assertCityScope()`, which allows only stores whose city is in the list (`403 CITY_OUT_OF_SCOPE`).

6. Route summary

All routes are under `/v1/admin`, need an ADMIN token, and pass through AdminGuard. Every POST returns **200**.

#	Method	Path	Allowed roles	Purpose
1	GET	`/admin/me`	READ	Who am I, my role and cities
2	GET	`/admin/onboarding/pipeline`	READ	Store pipeline list with counts
3	GET	`/admin/stores/:storeId`	READ	Full store review page data
4	GET	`/admin/documents`	READ	Document verification queue
5	POST	`/admin/documents/:id/verify`	VERIFY_DOCUMENTS	Mark a document verified
6	POST	`/admin/documents/:id/reject`	VERIFY_DOCUMENTS	Reject a document with a reason
7	POST	`/admin/stores/:storeId/start-review`	APPROVE_STORES	SUBMITTED → UNDER_REVIEW
8	POST	`/admin/stores/:storeId/approve-live`	APPROVE_STORES	Take the store LIVE
9	POST	`/admin/stores/:storeId/approve-limited`	APPROVE_STORES	Soft launch (LIMITED)
10	POST	`/admin/stores/:storeId/request-changes`	APPROVE_STORES	Send back to the partner (DRAFT)
11	POST	`/admin/stores/:storeId/reject`	APPROVE_STORES	Reject the application
12	POST	`/admin/stores/:storeId/suspend`	SUSPEND_STORES	Suspend a running store
13	POST	`/admin/stores/:storeId/reinstate`	REINSTATE_STORES	Lift a suspension

7. Route details — read routes

GET **`/v1/admin/me`**

Roles READ (any active admin except SALES_AGENT)

Handler `AdminController.me → AdminService.me`

Returns the logged-in admin and the cities they cover. The admin console calls this on load to show the name, role and city in the header.

```
200 { "user": { "id", "phone", "fullName" },
      "role": "FOUNDER",
      "cities": [ { "id", "name" } ],
      "allCities": true }
```

GET /v1/admin/onboarding/pipeline

Roles	READ
Handler	`AdminService.pipeline`
Query	`status?` one of DRAFT, SUBMITTED, UNDER_REVIEW, LIVE, LIMITED, PAUSED, SUSPENDED, REJECTED, CLOSED · `vertical?` EAT, GROCERY, PHARMACY, LOCAL_STORES · `q?` search text, max 80 characters

Lists stores for the onboarding pipeline, newest first, capped at 200 rows. `q` searches the store name and legal name (case-insensitive), the phone number, and the start of the PIN code. `counts` holds the number of stores in each status across the admin's whole scope, ignoring the current filters; these feed the status tabs.

```
200 { "counts": { "SUBMITTED": 8, "LIVE": 6, … every status … },
      "stores": [ { "id", "name", "vertical", "storeType", "status", "readinessScore",
                    "pincode", "phone", "createdAt", "updatedAt",
                    "city": { "name" }, "zone": { "name" },
                    "onboarding": { "status", "channel", "currentStep", "completedSteps",
                                    "flowVersion", "submittedAt" },
                    "docs": { "required", "verified", "pending", "rejected" } } ] }
```

`docs` counts only the latest upload of each document type. `required` depends on the store type (`onboarding/flows.ts → requiredDocuments`).

GET /v1/admin/stores/:storeId

Roles	READ
Handler	`AdminService.storeDetail`
Checks	storeId must be a UUID · store must exist (`404 STORE_NOT_FOUND`) · city scope

Everything the store review page needs, in one call. It also refreshes the readiness score and works out what blocks LIVE and LIMITED approval.

```
200 { "store":      { "id", "name", "status", "vertical", "storeType", "rejectedReason", "liveAt",
"createdAt" },
      "onboarding": { …full onboarding view: steps, answers, readiness… } | null,
      "members":    [ { "role", "isActive", "id", "phone", "fullName", "lastLoginAt" } ],
      "documents":  [ { "id", "type", "label", "status", "required", "documentNumber", "issuedAt",
                        "expiresAt", "fileHash", "rejectionReason", "reviewedBy", "reviewedAt",
                        "createdAt", "viewUrl", "contentType" } ],
      "consents":   [ { "type", "version", "state", "by", "ip", "channel", "acceptedAt" } ],
      "timeline":   [ { "step", "action", "actorType", "meta", "at" } ],      // last 50
      "adminActions": [ { "type", "reason", "params", "by", "at" } ],       // last 20
      "approval":   { "live": [ "problem", … ], "limited": [ … ] } }        // empty = allowed
```

`viewUrl` is a signed link (`/v1/files/raw?token=…`) that is valid for 15 minutes, so admins can open the uploaded file without exposing storage.

GET /v1/admin/documents

Roles	READ
Handler	`AdminService.documentQueue`
Query	`status` = PENDING (default; also includes UNDER_REVIEW) · VERIFIED · REJECTED

The document verification queue, oldest first, capped at 200. Each row says whether it is `superseded`, meaning the partner has uploaded a newer file of the same type. Only the newest file is meant to be reviewed; older ones are kept for audit.

```
200 [ { "id", "type", "label", "status", "required", "superseded", "documentNumber", "expiresAt",
        "createdAt", "viewUrl", "contentType",
        "store": { "id", "name", "status", "city", "legalName" } } ]
```

8. Route details — document review

POST /v1/admin/documents/:id/verify

Roles	FOUNDER, HEAD_OF_OPS, COMPLIANCE
Handler	`AdminService.reviewDocument (VERIFY)`
Body	`{ documentNumber?: string ≤ 40, expiresAt?: date }` (both optional; the number is stored trimmed and upper-cased)

- Document must exist (`404 DOCUMENT_NOT_FOUND`) and be in the admin's city scope.
- Status must be PENDING or UNDER_REVIEW (`409 DOCUMENT_ALREADY_REVIEWED`).
- If `expiresAt` is in the past, the request fails with `422 DOCUMENT_EXPIRED` ("reject it instead").
- On success, status becomes VERIFIED, and the reviewer and time are saved. An audit log entry and a `document.verified` event are written, then the store's readiness score is recalculated.

```
200 { "id": "…", "status": "VERIFIED" }
```

POST /v1/admin/documents/:id/reject

Roles	FOUNDER, HEAD_OF_OPS, COMPLIANCE
Handler	`AdminService.reviewDocument (REJECT)`
Body	`{ reason: string }`, 5–300 characters (required; the partner sees it)

- Same existence and scope checks as verify.
- Allowed from PENDING, UNDER_REVIEW or VERIFIED, so a verified document can later be rejected.
- On success, status becomes REJECTED and the reason is saved. An audit log entry and a `document.rejected` event (which carries the reason) are written. The partner's app updates live, and readiness is recalculated.

```
200 { "id": "…", "status": "REJECTED" }
```

9. Route details — store lifecycle

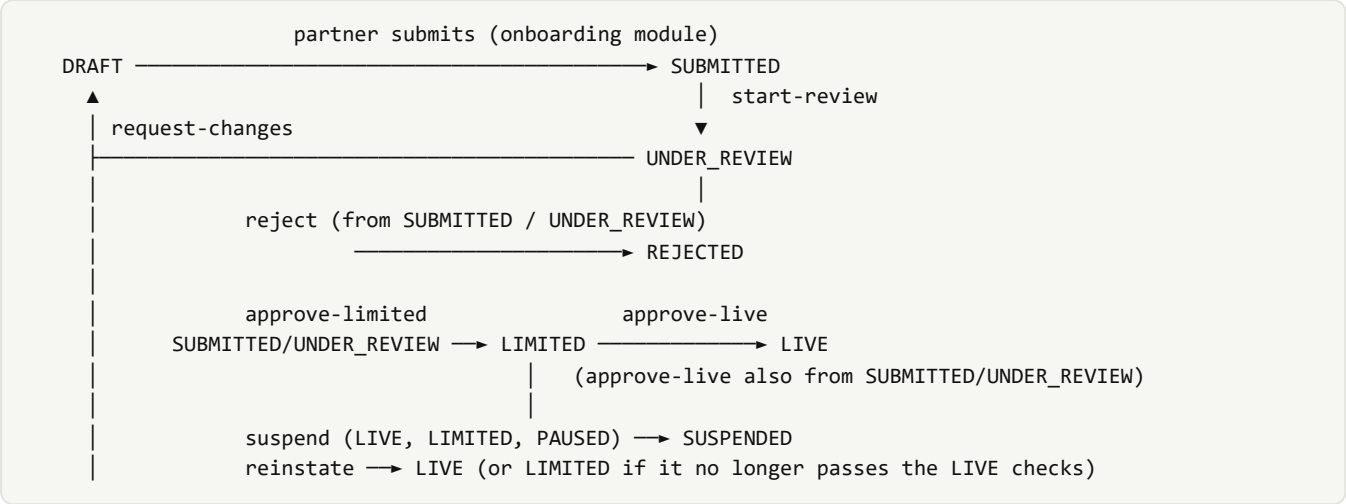
All seven actions share one implementation, `StoreLifecycleService.transition()`. After a store is submitted, this is the **only** code that changes its status. Each action has the same shape:

Params	`storeId` (UUID)
Body	`{ reason?: string ≤ 500 }` (start-review takes no body)
Response	`200 { "storeId", "from": "SUBMITTED", "to": "LIVE" }`

Route (POST .../stores/:storeId/...)	From status	To	Reason	Extra effects
`start-review`	SUBMITTED	UNDER_REVIEW	no	Partner sees "our team is reviewing".
`approve-live`	SUBMITTED, UNDER_REVIEW, LIMITED	LIVE	no	Runs the LIVE checks (§11); `422 APPROVAL_BLOCKED` with `details.problems` if any fail. Sets `liveAt` on first go-live, onboarding → APPROVED, writes an APPROVE_STORE admin action, emits `onboarding.approved`.
`approve-limited`	SUBMITTED, UNDER_REVIEW	LIMITED	no	Runs the LIMITED checks (§11). Otherwise the same as approve-live.
`request-changes`	SUBMITTED, UNDER_REVIEW	DRAFT	**yes**	Onboarding → NEEDS_CHANGES at step REVIEW. Store closed, with the reason shown to the partner.
`reject`	SUBMITTED, UNDER_REVIEW	REJECTED	**yes**	Saves `rejectedReason`. Onboarding → REJECTED, store closed, REJECT_STORE admin action.
`suspend`	LIVE, LIMITED, PAUSED	SUSPENDED	**yes**	Store forced CLOSED with the reason. SUSPEND_STORE admin action. Partner sees a red banner.
`reinstate`	SUSPENDED	LIVE or LIMITED	**yes**	Goes back to LIVE only if the store still passes the LIVE checks; otherwise it goes to LIMITED.

- Wrong starting status → `409 STORE_INVALID_TRANSITION` (details include the current status).
- Reason required but missing or under 5 characters → `400 REASON_REQUIRED`.
- Unknown store → `404 STORE_NOT_FOUND`. Out-of-scope city → `403 CITY_OUT_OF_SCOPE`.

10. Store lifecycle state machine



PAUSED and CLOSED exist in the schema but no admin route sets them yet. PAUSED is listed as a valid starting point for suspend.

11. Approval checks (LIVE / LIMITED)

`StoreLifecycleService.approvalProblems(storeId, target)` returns a list of human-readable problems. An empty list means the approval is allowed. The store review page shows these lists before the admin clicks anything.

Check	LIVE	LIMITED (soft launch)
Location	City, map pin (lat/lng) and address line 1 must all be set.	
Readiness score	at least 80	at least 60
Required documents	Every required document must be **VERIFIED**.	Every required document must be **uploaded**, and the latest one must not be REJECTED or EXPIRED.
Pharmacy	Needs an active registered pharmacist who has given consent.	Not allowed: pharmacies cannot soft-launch, they must go LIVE.

12. What every write records

All admin writes run in one database transaction, so either everything below is saved or nothing is:

Record	Table	When
Status change	`stores` / `partner_documents`	Always
Audit log (before/after, IP)	`audit_logs`	Every document review and every lifecycle action
Onboarding timeline event	`onboarding_events`	Every lifecycle action on a store that has an onboarding session
Admin action	`admin_actions`	approve-live, approve-limited (APPROVE_STORE), reject (REJECT_STORE), suspend (SUSPEND_STORE)

Domain event (outbox)	`outbox_events` → Socket.IO	`store.status_changed`, `onboarding.approved`, `document.verified`, `document.rejected`. The relay pushes these to the partner's open screens within about 0.5 s.

13. Error codes

Code	HTTP	Meaning
`AUTH_REQUIRED`	401	No bearer token
`TOKEN_INVALID` / `SESSION_REVOKED`	401	Expired or bad token / logged out
`WRONG_APP`	403	Token was issued for another app (e.g. PARTNERS)
`NOT_AN_ADMIN`	403	No active `admin_users` row (at login or on any route)
`ROLE_NOT_ALLOWED`	403	The admin's role can't do this action
`CITY_OUT_OF_SCOPE`	403	Store is outside the admin's cities
`VALIDATION_FAILED`	400	Bad query/body/UUID (field errors in `details`)
`REASON_REQUIRED`	400	Lifecycle action needs a reason of 5+ characters
`STORE_NOT_FOUND` / `DOCUMENT_NOT_FOUND`	404	Unknown ID
`STORE_INVALID_TRANSITION`	409	Store is not in a status this action starts from
`DOCUMENT_ALREADY_REVIEWED`	409	Verifying a VERIFIED document, or reviewing a REJECTED or EXPIRED one
`DOCUMENT_EXPIRED`	422	Verify with an expiry date in the past
`APPROVAL_BLOCKED`	422	LIVE/LIMITED checks failed. `details.problems` lists why.
`OTP_RATE_LIMITED` / `OTP_RESEND_TOO_SOON` / `OTP_TOO_MANY_ATTEMPTS`	429	Login limits (§3)

14. Database tables used

Model (table)	Used for
`AdminUser` (admin_users)	`userId` (unique link to users), `role`, `cityIds` (empty = all), `isActive`. Read by AdminGuard on every request.
`Store` (stores)	status, openState, closedReason, rejectedReason, liveAt, readinessScore, cityId
`OnboardingSession` / `OnboardingEvent`	Pipeline columns, store timeline, NEEDS_CHANGES / APPROVED / REJECTED
`PartnerDocument`	Queue and review: status, reviewedById, reviewedAt, rejectionReason, documentNumber, expiresAt, fileUrl
`AdminAction` (admin_actions)	adminId, storeId, type (APPROVE_STORE, REJECT_STORE, SUSPEND_STORE …), params {from,to}, reason
`AuditLog` (audit_logs)	actorUserId, actorType ADMIN, entity, entityId, action STATUS_CHANGE, before, after, ip

`ConsentRecord`, `PartnerMember`, `City`	Shown on the store review page

15. Admin web pages → API routes

Frontend: `apps/admin-web` (React + Vite) at `http://localhost:5174`. It stores the session in localStorage under the key `dz-admin-auth`.

Page (URL)	File	API calls
Login `/login`	`screens/Login.tsx`	`POST /auth/otp/request`, `POST /auth/otp/verify` (app ADMIN)
Header (all pages)	`screens/Layout.tsx`	`GET /admin/me`, `GET /admin/documents?status=PENDING` (badge count), `POST /auth/logout`
Pipeline `/`	`screens/Pipeline.tsx`	`GET /admin/onboarding/pipeline` (status tabs, vertical filter, search)
Document queue `/documents`	`screens/Documents.tsx` + `components/DocReview.tsx`	`GET /admin/documents?status=…`, `POST /admin/documents/:id/verify|reject`
Store review `/stores/:storeId`	`screens/StoreReview.tsx`	`GET /admin/stores/:storeId`, `POST /admin/stores/:storeId/<action>`, document verify/reject

16. Seeded admin accounts (dev)

Created by `packages/db/prisma/seed.ts` (`pnpm db:seed`). In dev, log in with the phone number and the `devCode` the OTP request returns. **These are development accounts only.**

Phone	Name	Role	Cities
+91 99999 00001	Founder	FOUNDER	All
+91 99999 00002	Bengaluru City Ops	CITY_OPS	Bengaluru only
+91 99999 00003	Compliance	COMPLIANCE	All
+91 99999 00004	Finance	FINANCE	All (read only)

17. Tests

- **API suite `e2e-admin.mjs`: 30/30 passing** (last run 19 Oct 2026). It covers:
  - A partner token on `/admin` → 403 WRONG_APP; `/admin/me`.
  - Role limits: FINANCE can read the pipeline but can't verify (403); COMPLIANCE can't reinstate (403).
  - CITY_OPS sees an in-city submitted store; the document queue lists it.
  - Approve LIVE blocked before documents are verified.
  - Start review, and a second start → 409.
  - Document reject: a missing reason → 400; rejecting with a reason.
  - Request changes: a missing reason → 400; → DRAFT; the partner sees NEEDS_CHANGES with the reason, re-uploads and resubmits.
  - Document verify, and a second verify → 409.

    - Approve LIVE: the partner is LIVE with 100 probation orders and can open.
    - Suspend: a missing reason → 400; the suspended store is closed and can't reopen.
    - FOUNDER reinstates → LIVE.
    - The partner receives the live events, and the admin action log is written.
- **Not covered by tests yet:** `approve-limited`, store `reject`, a city-scoped admin opening an out-of-scope store (403), and a non-admin number trying to log in with `app: ADMIN`.
- **Browser test `ui-admin.mjs`: passing.** A partner submits, an admin starts review and approves, and the partner's open page updates live.

These test scripts currently live in the session scratchpad, not in the repository. Moving them into the repo is part of Step 14 (hardening).

18. Known gaps and notes

City-scope mismatch for stores with no city. For city-scoped admins (e.g. CITY_OPS), the pipeline and document queue *include* stores whose `cityId` is empty (early drafts). But store detail, document review and every lifecycle action *reject* those same stores with `403 CITY_OUT_OF_SCOPE`. A scoped admin can see such a row but gets an error on opening it. Fix: either drop `cityId: null` from the list scope, or allow it in `assertCityScope`. A one-line change either way, but it's a policy decision.

SALES_AGENT has no access. The role exists in the schema (and `OnboardingSession` has a `salesAgent` relation), but it is in no `ROLES` group, so every admin route returns `403 ROLE_NOT_ALLOWED`. That's fine until a sales-agent flow is built.

No pagination. The pipeline and document queue return at most 200 rows. That's fine for launch volume; add cursor paging before scaling.

Lifecycle POSTs are not idempotent. They don't use `@RequireIdempotency()`. A double-click is still safe, because the second call fails with `409 STORE_INVALID_TRANSITION`, but it shows the admin an error.

Not built yet: the other AdminActionType values (warnings, throttling, ETA overrides, penalty waivers …), admin user management, settlements and order operations. These depend on later steps (Orders, Payouts).